Model Compression and Multiobjective Evaluation of RF-DETR for Parking Perception on a Laptop GPU

Sugyeong Kim[1], Myungkyun Kim[2], Sua Kim[1], Hyo Jin Park[3], Jae Wook Jeon[3]

[1] School of Mechanical Engineering, Sungkyunkwan University

[2] Department of Electronic and Electrical Engineering, Sungkyunkwan University

[3] Department of Electrical and Computer Engineering, Sungkyunkwan University

Abstract: We evaluated 26 compression candidates derived from one RF-DETR Segmentation Large model for f ront-camera parking perception. The comparison used 4,466 images split into training, validation, and a fixed c omparative benchmark. We measured accuracy, latency, and TensorRT engine size on an RTX 4050 Laptop GP U for candidates using precision conversion, pruning, 2:4 sparsity, resolution reduction, and combined methods. After accuracy screening, three-objective Pareto analysis retained C01, which combines decoder reduction with FP16, and R01, which combines reduced resolution with FP16. C01 reached Mask AP 0.6029, with 44.79% lo wer median latency and a 49.95% smaller engine than the FP32 baseline. At 24.010 ms, R01 was the fastest p assing candidate, although its accuracy margins were smaller. Much of the acceleration came from FP16; accur acy losses and execution configurations constrained the sparse and aggressive FP8 candidates. The results identi fy a balanced candidate and a speed-prioritized candidate for further deployment validation, and show why stati c compression metrics require target-GPU measurements. Reuse of the benchmark limits claims about independ ent-environment generalization.

Keywords: RF-DETR, TensorRT, model compression, autonomous parking, instance segmentation

1 Introduction

Front-camera parking perception requires boundi ng boxes and masks to represent parking spaces a nd boundaries. Deployment also depends on infere nce latency and memory demands. We evaluate th ese outputs and execution costs for compression c andidates derived from a common RF-DETR segm entation model on an RTX 4050 Laptop GPU.

TensorRT latency depends on graph fusion, preci sion fallback, kernel selection, and memory transfe rs, so parameter counts and theoretical operation c ounts alone cannot predict it. We therefore examin e which changes to a trained model reduce measur ed cost while preserving detection and segmentatio n accuracy.

Research Questions and Scope

The comparison addresses three questions. RQ1: How do the compression methods change measure d latency and TensorRT engine size on the laptop GPU? RQ2: How do BBox AP, Mask AP, and se mantic mIoU change as inference becomes faster? RQ3: After accuracy screening and Pareto analysis, which candidates support balanced or speed-prioriti zed deployment?

Our contribution is a fixed-baseline comparison linking candidate configurations to measured result s, with matched-precision and class-level analyses. The scope is front-camera model inference on one laptop GPU, not complete ROS2 latency or parkin g success. The benchmark is reused for candidate comparison rather than reserved as an untouched c onfirmatory test.

2 Related Work

2.1 DETR and RF-DETR

DETR formulates detection as set prediction wit h object queries and bipartite matching [2]. DINO improves query training and initialization [3], and Mask DINO extends the framework to instance se gmentation [4]. RF-DETR supplies the bounding-b ox and mask predictions used in this study [1,13].

2.2 Pruning and Sparse Execution

Unstructured pruning removes individual weights [5], whereas structured pruning removes layers or hidden dimensions [6,12]. Only supported sparse k ernels can exploit irregular zero patterns. NVIDIA 2:4 sparsity requires two zero weights per group o f four; acceleration additionally requires TensorRT to select sparse tactics [9].

2.3 Reduced Precision and Input Resolution

Reduced precision can lower storage and executi on cost, but its benefits depend on calibration, sen sitive layers, and hardware support. ONNX Quanti zeLinear/DequantizeLinear (Q/DQ) nodes alone do not establish low-precision execution [10,14]. Smo othQuant and AWQ were primarily validated on la

rge language models [7,8]; their benefits cannot be assumed to transfer to our ModelOpt-based RF-DETR candidates. Lower input resolution reduces feature-map computation but may remove small-object or thin-boundary detail.

2.4 Research Gap and Positioning

Our previous work introduced a modular ROS2 framework integrating perception, decision-making, and vehicle communication [16]. We subsequently developed a state-based perpendicular-parking system using front and rear camera segmentation, LiDAR, and bird's-eye-view geometry [17]. These studies provide the system context for parking perception. The present study examines compression of an RF-DETR perception model rather than evaluating complete parking maneuvers.

RF-DETR already explores an accuracy-latency Pareto frontier through weight-sharing neural architecture search [1]. Our distinction is the comparison of post-training compression methods applied to one trained parking model under common data and laptop-GPU conditions, with engine size as a third objective. We do not claim architecture search, runtime evaluation, or Pareto analysis itself as a new method.

3 Dataset and Split Protocol

3.1 Dataset Composition

We use a self-collected dataset comprising 4,466 front-camera images of parking scenes. Bounding boxes and polygon masks are annotated for three classes—out_line, parking_lot, and parking_space—and provided in COCO instance segmentation format [11]. The available records do not specify the camera model, source-video resolution, annotation tool, boundary rules, or review history.

3.2 Session-Based Splitting

We recovered capture times and frame indices from filenames to group adjacent video frames into sessions. A new session began when the time gap exceeded 2.0 s or the frame index reset. The observed gaps reached 1.411 s within sessions and were at least 5.010 s between sessions. All images from an identifiable session were kept in the same split.

The 1,461 traceable images belonged to seven timestamp sessions, which we divided 3/2/2 among training, validation, and the benchmark. All 3,005 untraceable images were assigned to training. The audit found no byte-identical images or identifiable sessions shared across splits, and no COCO reference errors. Capture-related correlations involving the untraceable training images nevertheless remained possible.

3.3 Final Splits and Their Roles

Table 1. Session-based dataset split

Split	Count	Share and role
Train	3,625	81.17%; 3 sessions Training, recovery, final calibration
Validation	404	9.05%; 2 sessions Checkpoint selection, sensitivity search
Benchmark	437	9.79%; 2 sessions Repeated candidate comparison

Table 1 defines the split roles. Final-candidate calibration uses training images; temporary FP8 sensitivity models use validation data (Section 4.4). Benchmark images are excluded from calibration and protected-block selection. Figure 1 shows image-level condition prevalence; the mean absolute differences from training are 15.33 percentage points for validation and 3.89 for the benchmark.

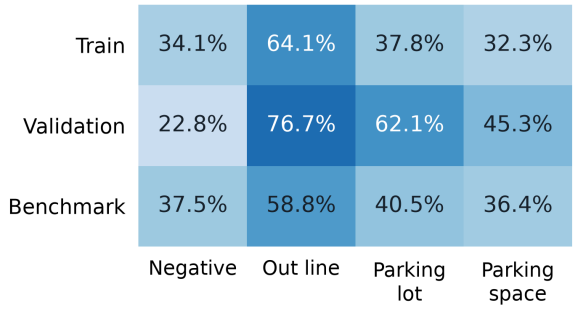


Fig. 1. Image-level condition prevalence by split. Negative denotes images without target objects. Class-presence conditions are not mutually exclusive.

4 Design of Deployment Candidates

4.1 Baseline Model

The baseline uses Segmentation Large from RF-DETR 1.8.1 with a 504×504 input [13]. Starting from pretrained RF-DETR weights, we train on 3,625 images with batch size 2, gradient accumulation 8, effective batch size 16, learning rate 1×10^{-4}, encoder learning rate 1.5×10^{-4}, weight decay 1×10^{-4}, multi-scale training, exponential moving average (EMA), and seed 42. The maximum is 25 epochs. Early stopping monitors validation mAP with patience 6 and a minimum delta of 0.001; training ends at epoch 18. The final checkpoint_best_total.pth selects the better EMA or regular checkpoint according to validation performance. Baseline and recov

ery training use an RTX 5060 Ti, whereas deployment evaluation uses a separate RTX 4050 Laptop GPU.

B01 denotes FP32 TensorRT execution of the selected checkpoint. All candidates derive from these weights, directly or through recovery. Configuration and result provenance use the pinned repository revision in [15]. Accuracy, engine-build, and repeated-latency records are linked by engine and annotation SHA-256 hashes.

4.2 Compression Candidates

Table 2. Candidate families and configurations

Family	ID	Configuration
Baseline	B01–B03	B01: FP32; B02: FP16; B03: INT8 PTQ
Unstructured	U01–U03	Global magnitude pruning 10% / 30% / 50%
2:4 sparsity	M01–M02	M01: dense control; M02: sparse tactic; both FP16
Structured	S01–S04	S01/S02: remove 1/2 decoder layers; S03/S04: reduce FFN dimensions by 20%/40%
Combined	C01–C04	C01: S01+FP16; C02: S01+INT8 PTQ; C03/C04: S01+432/480 FP32
Resolution	R01–R03	R01: 432+FP16; R02: 480 FP32; R03: 384 FP32
Q/DQ	Q01–Q07	Q01: INT8-SQ; Q02: INT8 mixed; Q03: INT4; Q04–Q07: FP8 protection rules (Section 4.4)

Table 2 groups the 26 candidates into seven families. S01/S02 remove the final decoder layers and corresponding segmentation blocks. S03/S04 jointly prune linear1 outputs and biases and linear2 inputs in each decoder feed-forward network (FFN), retaining larger-magnitude units and aligning hidden dimensions to multiples of 32.

For B03, we use implicit INT8 post-training quantization (PTQ). The Q01–Q03 recipes instead use SmoothQuant, mixed INT8 precision, and INT4 weight-only quantization, respectively. Q04–Q07 differ in their FP8 block protection. The following sections describe recovery and quantization for these candidates.

4.3 Recovery Training and Sparse Pattern Preservation

For S01, S02, and M01, recovery starts from the structurally modified or 2:4-sparsified weights. We reinitialize the optimizer and scheduler, train on the training split, and use validation for checkpoint selection and early stopping. The common recipe allows up to 10 epochs with learning rate 1×10^{-5}, encoder learning rate 1.5×10^{-5}, effective batch size 16, and weight decay 1×10^{-4}. It uses a cosine scheduler, one warm-up epoch, EMA decay 0.993, patience 3, and seed 42. S01 and S02 finish at epochs 6 and 8, respectively. For M01, metadata records 10 completed epochs, but the detailed recovery report is missing from the pinned revision. Its execution provenance therefore remains incomplete.

C01–C04 reuse the recovered S01 weights. M02 uses the M01 weights with sparse tactics enabled; gradient masking and mask reapplication after each optimizer step preserve the 2:4 pattern. We do not recover S03 after its static exclusion. S04 has incomplete recovery and is retained for diagnostic analysis. Precision-only, resolution-only, and PTQ candidates receive no additional training.

4.4 Quantization and FP8 Sensitivity Search

B03 uses IInt8EntropyCalibrator2 with 128 training images and allows FP16 fallback. To select the images, we sort filenames, shuffle them with seed 42, and take the calibration subset. We then convert images to RGB, resize them to the input dimensions, and apply ImageNet normalization. The actual 128-image manifest identifies the set used for calibration. B03 follows implicit PTQ, whereas Q-family exports include explicit ONNX QuantizeLinear and DequantizeLinear (Q/DQ) nodes.

Final FP8 candidates use 128 training images for calibration, seed 42, and 504×504 inputs. Temporary sensitivity models use 64 validation images, selected after sorting with seed 42. AP sensitivity is evaluated on the first 150 sorted COCO image IDs; Kullback–Leibler divergence (KLD) uses 48 validation images selected with seed 7. The benchmark is excluded from both stages.

All FP8 variants use ModelOpt FP8_DEFAULT_CFG with Conv2d quantizers disabled. Q04 adds no protection. Q05 quantizes one of 13 block groups at a time and protects the 11 whose larger BBox/Mask AP drop exceeds 0.001: final heads, decoder layers 0–4, backbone projector, backbone layer groups 00–02, 03–05, and 06–08, and encoder output/heads. Q07 protects only final_heads and decoder_layer_4, the two highest-ranked groups. Protection disables quantizers; it does not quantize only the protected blocks.

Q06 protects six of 21 blocks, rounding up the top 25% by KLD: backbone projector, final heads, encoder output/heads, and backbone layers 2, 9, an

d 0. The 50 highest-confidence reference queries a re aligned by Hungarian matching with box-plus-cl ass L1 cost. Scores sum Bernoulli KLD for class outputs and stride-4 masks. AP and KLD use diffe rent block partitions. FP8 export uses opset 19.

Table 3. Additional protection strategies for FP8 candidates

ID	Selection rule	Protected units
Q04	None	0
Q05	AP drop > 0.001	11 / 13 groups
Q06	Top 25% by KLD	6 / 21 blocks
Q07	Top 2 by AP sensitivity	2 / 13 groups

Disabling a quantizer does not cast a block to F P16. Q05 engine inspection confirms FP32 inputs/ outputs in the class/bbox heads and decoder layer 4 FFN, but FP8 inputs and an FP8 GEMM tactic in the backbone layer 11 FFN. Q05 is therefore a protected mixed-precision FP8 candidate. These ob servations do not establish the precision of other c andidates or the fraction of FP8 operations from q uantizer counts.

4.5 Evaluation Workflow

Figure 2 summarizes candidate construction, opti onal recovery, artifact reinspection, and target-GPU measurement. Section 5 defines measurement proc edures before the accuracy gate and Pareto criteria.

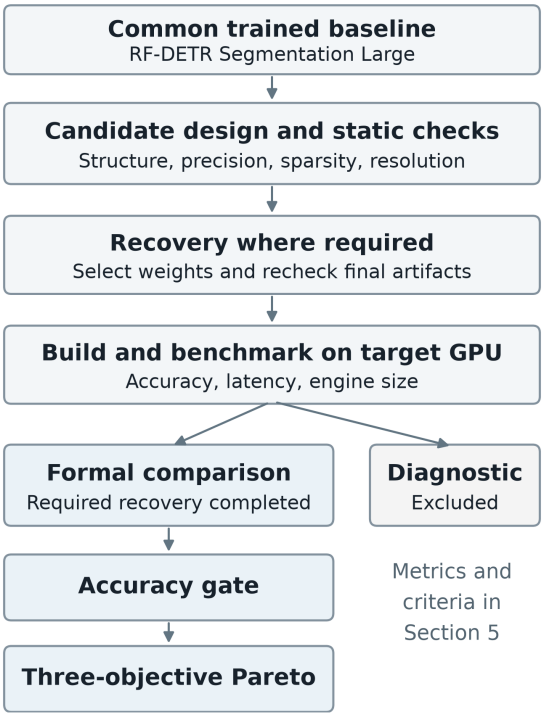


Fig. 2. Candidate generation and evaluation workflow. Recovery is applied only when required.

5 Experimental Setup and Evaluation Meth ods

5.1 Execution Environment

All engines are built and measured on the lapto p specified in Table 4.

Table 4. Execution environment

Component	Configuration
GPU	NVIDIA GeForce RTX 4050 Laptop GP U, 6 GB
CPU	AMD Ryzen 7 7735HS, 8-core/16-thread
RAM	16 GB
OS	Ubuntu 22.04
CUDA	12.8
TensorRT	10.16.1.11
Power	AC power, performance mode
CC / Driver	8.9 / 580.173.02

Building and measurement use AC power, a per formance platform profile, and the performance C PU energy preference. Load is stabilized before m easurement; temperatures and clocks are logged be fore and after, without fixed clocks or a recorded GPU power limit (N/A). Versions are RF-DETR 1. 8.1, PyTorch 2.10.0, torchvision 0.25.0, ModelOpt 0.46.0, and ONNX 1.22.0 [15].

5.2 Accuracy Evaluation

We evaluate accuracy on the same 437 benchma rk images. BBox AP and Mask AP use COCO Io U thresholds from 0.50 to 0.95 at 0.05 intervals [1 1]. Postprocessing retains the top 200 query/class s cores without non-maximum suppression (NMS). P redictions at confidence 0.001 then enter the COC O evaluator with maxDets=100; this evaluator limi t is separate from output selection. Only candidate s with valid engines and completed recovery, wher e required, enter formal comparisons.

Region-level evaluation uses binary instance mas ks from RF-DETR 1.8.1 PostProcess [13]. Selecte d mask logits are resized to the original image di mensions using bilinear interpolation with align_co rners=False and binarized at logit > 0. Predicted masks with confidence at least 0.25 are combined by class-wise union; ground-truth masks are combi ned in the same way. For each class, intersection and union pixel counts are accumulated over the e ntire benchmark before computing IoU. Semantic mIoU is the mean over classes with nonzero union. It is not a simple average of per-image IoU. Overl

apping regions from different classes are evaluated independently for each class. AP50, AP75, size-specific and class-specific AP, and AR100 are retained as supplementary metrics.

5.3 Runtime Performance and Repeatability

Table 5. Stage 2 measurement protocol

Item	Fixed setting
Accuracy	437 benchmark images
Runtime	Seed 42; fixed 32 images; batch 1; confidence 0.25
Repeats	3 repeats of 20 warm-up + 200 timed iterations
Timing scope	Preprocessing, host-to-device (H2D) transfer, TensorRT, postprocessing, device-to-host (D2H) transfer; excludes disk reading and decoding
Latency	Median, mean, P95, P99, IQR, FPS
Repeatability	CV of repeat medians; remeasure if > 5%
30 FPS reference	Median ≤ 33.33 ms; P95 as a warning metric

The runtime samples and repeats are specified in Table 5. We start timing with RGB PIL images already in memory, so the measurements exclude disk reading, decoding, ROS2 communication, and control. Each iteration is timed with perf_counter, with GPU synchronization immediately before and after execution.

We calculate median and P95 latency from the pooled set of 600 timed measurements. For repeatability, we divide the sample standard deviation of the three repeat medians by their mean to obtain CV. Engine size is the file size in bytes divided by 2^{20} (MiB). The 33.33 ms median reference represents the frame period at 30 FPS. It does not establish sustained throughput or every-frame timing; P95 describes tail latency.

5.4 Stage Criteria and Accuracy Preservation

Stage 1 checks artifacts and recipes without accuracy or latency measurements. The 5% reduction requirement applies to structural, resolution, and unstructured-pruning candidates: at least one of ONNX size, graph node count, or dense multiply–accumulate operations (MACs) must decrease by at least that amount relative to B01. We retain B01 as the baseline and B02 and B03 as precision controls with defined engine-build recipes. Q01–Q07 require explicit Q/DQ nodes consistent with their quantization reports; B03 uses implicit INT8 PTQ and does not require those nodes. For 2:4 candidates, we verify the weight pattern and applicability of sparse execution.

In Stage 2, candidates passing Stage 1 are converted to TensorRT engines in the environment above and evaluated for accuracy on the 437-image benchmark and repeated inference performance. The accuracy-preservation gate permits decreases relative to B01 of no more than 0.01 in BBox AP, 0.01 in Mask AP, and 0.02 in semantic mIoU. These are engineering deployment criteria defined before the experiment to preserve detection and segmentation performance within specified tolerances, not tests of statistical significance or equivalence.

5.5 Pareto Criteria

We apply Stage 3 to candidates that pass the accuracy gate, using Mask AP, median latency, and engine size as the objectives. Higher Mask AP is preferred; lower latency and engine size are preferred. A candidate is dominated when another is no worse on any objective and better on at least one. BBox AP, semantic mIoU, P95, and GPU memory support interpretation but do not enter the final dominance test.

6 Results and Discussion

6.1 Stage 1 Static Screening

Stage 1 retained 22 of 26 candidates for TensorRT evaluation. U01–U03 increased zero sparsity but missed the 5% dense-artifact reduction criterion, as did S03. M01/M02 met the 2:4 pattern in all 150 eligible operations, and Q01–Q07 passed the ONNX checker with explicit Q/DQ nodes. We used these checks to select candidates for measurement; acceleration remained to be tested.

The MAC/FLOP estimates cover resolvable Conv, MatMul, and Gemm shapes and are lower bounds on dense arithmetic. They exclude TensorRT fusion, transfers, preprocessing, and postprocessing.

6.2 Stage 2 Accuracy and Runtime Results

Q03 fails engine construction: the TensorRT 10.16.1.11 parser reports INVALID_NODE for a DequantizeLinear input interpreted as Float with block size 128. This export/parser incompatibility does not establish a general limitation of INT4. The other 21 candidates have engines and metrics; excluding diagnostic S04 leaves 20 formal candidates. B01 achieves BBox AP 0.7379, Mask AP 0.6026, and semantic mIoU 0.7120. Appendix A, Table A1 reports all measured candidates.

Removing one decoder layer and completing rec
overy preserves aggregate accuracy but yields a m
odest runtime reduction at FP32. S01 passes with
BBox AP 0.7390, Mask AP 0.6022, and mIoU 0.7
339; its median latency is 44.510 ms against 45.82
0 ms for B01. S02 removes two layers and reache
s 42.957 ms, but both AP losses exceed the limits
even after recovery. We therefore exclude the deep
er reduction from deployment selection despite its
shorter latency.

FP16 accounts for a large reduction in execution
cost: B02 lowers median latency by 43.75% and e
ngine size by 47.47% relative to B01, with a Mas
k AP change of −0.0009. INT8 B03 passes the gat
e but reduces median latency by only another 1.6
1%, while the B02/B03 engines occupy 67.9/68.0
MiB. We use B02 to assess the additional benefit
of FP16-based compression because these controls
share the trained model and 504×504 input. B03 a
llows FP16 fallback, so the INT8 label does not
mean every operation runs in that format. Its meas
ured advantage over FP16 is small in this configur
ation.

Against the FP16 control B02, C01 reduces med
ian latency by 1.85% and engine size by 4.73%. T
hese are the additional reductions beyond precision
conversion, rather than the larger change measured
from B01 to C01. Because decoder reduction and
recovery are applied together, their effects cannot
be separated here. Figure 3 therefore presents inde
pendent candidate comparisons; the reductions sho
uld not be added as contributions from individual
methods.

R01 trades lower Mask AP for shorter latency:
changing the input from 504×504 to 432×432 redu
ces median latency by 6.85% relative to B02. At
384×384, FP32 R03 has 29.32% lower median lat
ency than B01 but fails both AP criteria. Its media
n of 32.386 ms meets the 33.33 ms reference, wh
ereas P95 reaches 34.875 ms. We therefore assess
resolution choices using accuracy and tail latency
alongside the median; meeting the median referenc
e alone is insufficient.

Beyond 2:4 pattern compliance, the M02 logs sh
ow that TensorRT selected sparse tactics. Compare
d with dense-control M01, median latency falls by
0.49% and engine size by 8.11%, but both candida
tes lose more than 0.08 Mask AP and fail the gate.
M01 also has an export-equivalence failure for the
engine-source ONNX. Some loss is already presen
t in its PTH checkpoint, so export cannot explain
all of it, and the result should not be generalized t
o sparsity. Although sparse execution was achieve
d for M02, this pair offers no useful acceleration
with preserved accuracy. The recipe needs further
recovery and export validation before it can be ass
essed for deployment.

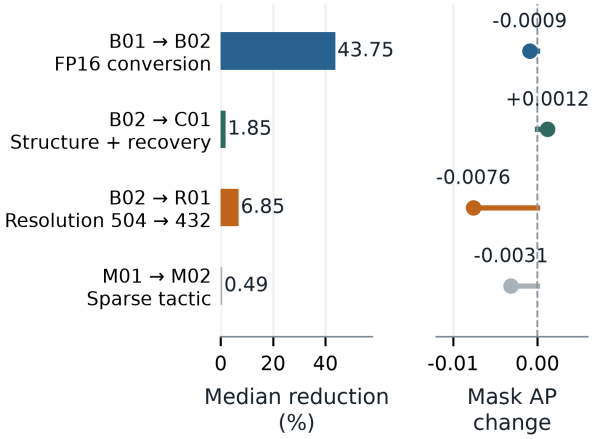


Fig. 3. Median latency reduction and Mask AP change for independent candidate pairs. Each row has a different reference, so reductions are not additive. M01 and M02 both fail the accuracy gate.

Q01/Q02 produce smaller engines of 47.8/43.4
MiB and median latencies near 25 ms, but their
Mask AP losses of 0.1283/0.1194 exceed the accur
acy limits. Among FP8 variants, Q05 alone passes.
Its AP-based protection preserves accuracy better i
n this comparison than the Q06 KLD and Q07 top
-two rules. However, the protected scopes and exe
cution precision also differ, so the sensitivity metri
c alone cannot explain that result. Q05 reduces me
dian latency by 5.43% and engine size by 13.54%
relative to B01, but C01 dominates it on the select
ed objectives.

The protection trade-off is visible in the executi
on records and Table A1. Q05's 43.333 ms median
and 111.8 MiB engine remain above the FP16 con
trol. By contrast, the less protected Q04 engine is
smaller but has BBox AP 0.0036 and Mask AP 0.
3702. These results support evaluating the exporte
d mixed-precision configuration as a whole, witho
ut inferring acceleration from the nominal FP8 for
mat or quantizer count.

Eleven formal candidates pass the gate and nine
fail (Table A1). Figure 4 locates the threshold viol
ations using changes relative to B01. Among passi
ng candidates, B03 has the highest BBox AP (0.74
22) and C02 the highest mIoU (0.7412), but C01
dominates both on the three objectives. S02 has th
e highest formal-candidate mIoU (0.7531) yet fails

both AP bounds. Decisions and differences use unr ounded values.

All 21 measured engines meet CV ≤ 5%; the m aximum is 3.8249% for R01. This repeatability ch eck does not establish statistical significance of sm all between-candidate differences such as 1.85% or 0.49%. Accuracy estimates likewise cover only tw o benchmark sessions, without session-level confid ence intervals.

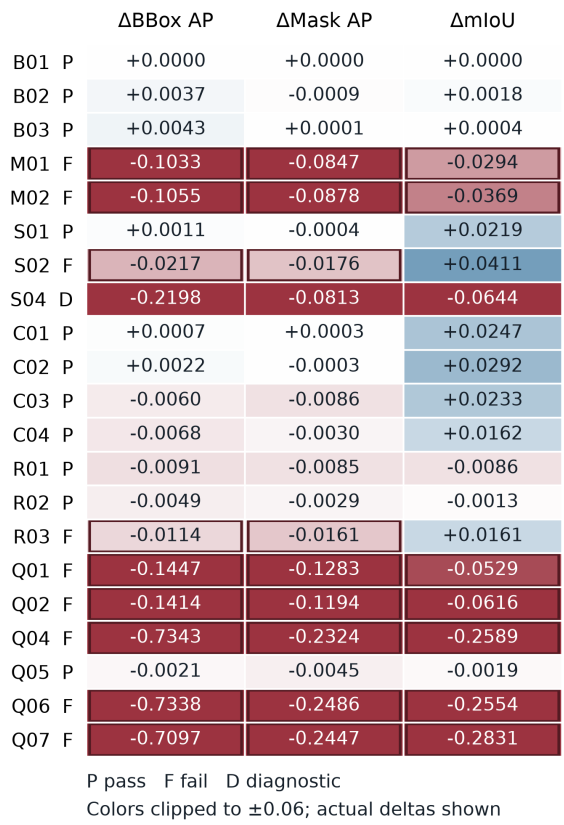


Fig. 4. Accuracy changes relative to B01 and gate decisions. Outlined cells violate the permitted lower bounds. D denotes diagnostic results excluded from formal comparison.

6.3 Pareto Analysis and Final Candidates

Table 6. Baseline, FP16 control, and final candidates

ID	Mask AP	Median (ms)	P95 (ms)	Engine (MiB)
B01	0.6026	45.820	48.562	129.3
B02	0.6017	25.775	28.130	67.9
C01	**0.6029**	**25.300**	**27.512**	**64.7**
R01	**0.5941**	**24.010**	**25.821**	**68.1**

Only C01 and R01 remain nondominated (Fig. 5). Table 6 compares them with the FP32 baseline and FP16 control. From unrounded values, C01 ha s 1.289 ms higher median latency, 0.0088 higher Mask AP, and a 3.32 MiB smaller engine than R0 1. The aligned plots are sorted by median latency, not an overall ranking.

Both candidates have median and P95 latency b elow 33.33 ms. C01 balances accuracy and engine size; R01 prioritizes speed. However, R01 exceeds the BBox/Mask AP gate thresholds by only approx imately 0.00094/0.00147, so its pass decision may change on new sessions.

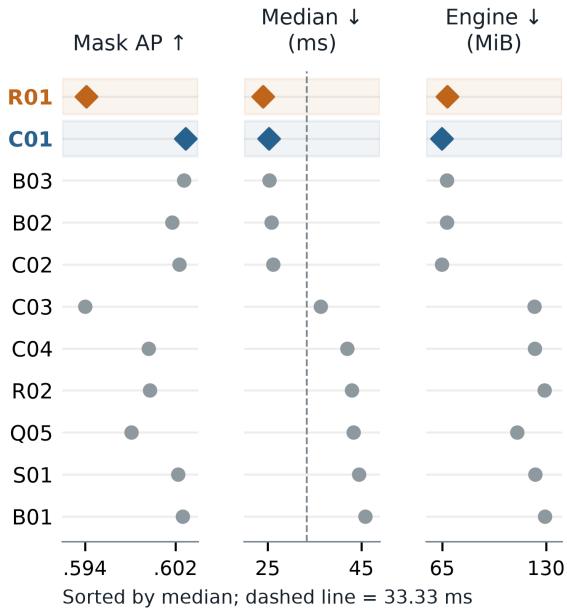


Fig. 5. Three-objective comparison of candidates passing the accuracy gate. C01 and R01 are nondominated. Mask AP, median latency, and engine size should be considered together within each row.

6.4 Class-Level Performance and Deployment I mplications

R01 passes the aggregate gate despite an approx imately 0.0225 drop in out_line Mask AP, compar ed with its mean drop of 0.0085. C01 also passes, although parking_space BBox AP falls by 0.0121. Figure 6 shows these class-level changes, and Tabl e A2 gives the absolute AP and semantic IoU valu es. The aggregate pass decisions therefore leave sp ecific boundary and parking-space losses to check before deployment. We report these losses alongsid e the decisions and keep the original gate unchang ed.

The low out_line result is already present in B0 1: its Mask AP is 0.2040, compared with 0.8086 f or parking_lot and 0.7954 for parking_space. C01 raises out_line semantic IoU from 0.4589 to 0.492 7 while slightly lowering Mask AP. These metrics assess different outputs: semantic IoU scores the o verlap after masks are combined by class, whereas AP evaluates individual instances. We therefore us

e both metrics to assess boundary perception; the gain in union overlap does not remove the instanc e-level loss.

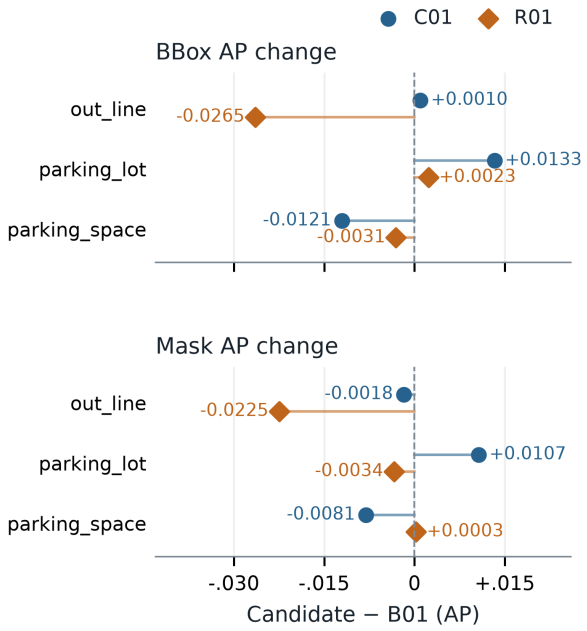


Fig. 6. Class-level AP changes for C01 and R01 relative to B01. BBox AP is shown above and Mask AP below, with a common horizontal scale.

6.5 Threats to Validity

Training uses one seed (42), so training variabili ty is unmeasured. All seven identifiable sessions c ome from one capture day; validation and benchm ark sets have two sessions each. Untraceable traini ng images, benchmark reuse, and the absence of a locked external holdout limit generalization. U01–U03 and S03 are not timed after static exclusion, so their rejection does not show that these method s can never accelerate inference.

The measurements apply to the specified laptop GPU, TensorRT version, and power settings. Logs taken before and after the runs cannot establish su stained thermal stability or fixed total graphics po wer. Most candidates allocate approximately 890–8 98 MiB at peak, so this metric distinguishes them poorly. M01 recovery provenance and export equiv alence also limit interpretation, as do missing cam era and annotation records. Validation under indep endent capture conditions, across multiple seeds an d other GPUs, and through the complete ROS2 pi peline remains necessary.

7 Conclusion

Within the evaluated set, we select C01 for bala nced deployment and R01 when latency has priorit y. Most of the measured latency reduction on this laptop GPU comes from FP16. Structural and reso lution changes add smaller trade-offs, while the sp arse and FP8 results show the need to check accur acy and actual execution before interpreting static compression as acceleration. The two selections re quire further validation: their class-level losses and R01's small gate margins must be checked on ind ependent sessions before deployment.

Acknowledgments

This work was supported by the Korea Institute for Advancement of Technology (KIAT) grant fun ded by the Ministry of Education (Project No. P00 30064, Advanced Industry Talent Training Bootca mp Program - Sungkyunkwan University (Future Mobility)).

References

1) I. Robinson, P. Robicheaux, M. Popov, D. Ramanan and N. Peri, "RF-DETR: Neural Architecture Search for Real-Time Detection Transformers," arXiv preprint arXiv:2511.09554, 2025.
2) N. Carion et al., "End-to-End Object Detection with Transf ormers," Proceedings of ECCV, pp. 213-229, 2020.
3) H. Zhang et al., "DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection," Proceedin gs of ICLR, 2023.
4) F. Li et al., "Mask DINO: Towards a Unified Transformer-Based Framework for Object Detection and Segmentation," Proceedings of CVPR, pp. 3041-3050, 2023.
5) S. Han, J. Pool, J. Tran and W. Dally, "Learning both Weig hts and Connections for Efficient Neural Networks," Advan ces in Neural Information Processing Systems, Vol. 28, 201 5.
6) H. Li, A. Kadav, I. Durdanovic, H. Samet and H. P. Graf, "Pruning Filters for Efficient ConvNets," Proceedings of IC LR, 2017.
7) G. Xiao et al., "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," Proceed ings of ICML, Vol. 202, pp. 38087-38099, 2023.
8) J. Lin et al., "AWQ: Activation-Aware Weight Quantization for On-Device LLM Compression and Acceleration," Procee dings of MLSys, Vol. 6, 2024.
9) NVIDIA, "Sparsity," TensorRT Developer Guide, https://doc s.nvidia.com/deeplearning/tensorrt/10.x.x/inference-library/io-formats-sparsity.html, accessed Sep. 5, 2026.
10) NVIDIA, "Working with Quantized Types," TensorRT Dev eloper Guide, https://docs.nvidia.com/deeplearning/tensorrt/la test/inference-library/work-quantized-types.html, accessed Se p. 5, 2026.
11) T.-Y. Lin et al., "Microsoft COCO: Common Objects in C ontext," Proceedings of ECCV, pp. 740-755, 2014.
12) H. Yang, H. Yin, M. Shen, P. Molchanov, H. Li and J. Ka utz, "Global Vision Transformer Pruning With Hessian-Awar e Saliency," Proceedings of CVPR, pp. 18547-18557, 2023.
13) Roboflow, "RF-DETR: Real-Time Object Detection and Se gmentation," GitHub repository, https://github.com/roboflow/ rf-detr, accessed Sep. 5, 2026.
14) NVIDIA, "PyTorch Quantization," TensorRT Model Optimi zer Documentation, https://nvidia.github.io/Model-Optimizer/ guides/_pytorch_quantization.html, accessed Sep. 5, 2026.
15) "ai-model-lightweighting," GitHub repository, https://githu b.com/ksk192830/ai-model-lightweighting, revision 46ecbc2a 2585f06ac76499dec16579c21048173a, accessed Sep. 8, 202 6.

16) S. Kim, Y. Woo and K. Kim, "Development of a Modular ROS2 Software Framework for Autonomous Vehicles," Proceedings of the 2025 KSAE Annual Autumn Conference and Exhibition, pp. 1896-1903, 2025.
17) S. Kim, M. Kang, M. Kim, H. J. Park and J. W. Jeon, "Enhancing Perpendicular Parking in a ROS2 Autonomous Driving Framework: A State-Based Multi-Sensor Approach," Proceedings of the Korea Information Processing Society Conference (ASK 2026), Vol. 33, No. 1, pp. 676-678, 2026.

Appendix A Complete Measured Results

Table A1. Accuracy and runtime results for all measured candidates

ID	BBox AP	Mask AP	mIoU	Median (ms)	P95 (ms)	Engine (MiB)	CV (%)	Gate
B01	0.7379	0.6026	0.7120	45.820	48.562	129.3	0.12	P
B02	0.7416	0.6017	0.7138	25.775	28.130	67.9	2.88	P
B03	0.7422	0.6027	0.7124	25.360	27.903	68.0	2.26	P
M01	0.6346	0.5180	0.6826	25.387	27.630	67.9	0.96	F
M02	0.6324	0.5148	0.6751	25.262	28.146	62.4	0.57	F
S01	0.7390	0.6022	0.7339	44.510	46.954	123.3	1.16	P
S02	0.7161	0.5850	0.7531	42.957	45.106	117.2	0.56	F
S04	0.5181	0.5214	0.6476	45.769	48.077	121.2	0.58	D
C01	**0.7386**	**0.6029**	**0.7367**	**25.300**	**27.512**	**64.7**	**2.91**	**P**
C02	0.7401	0.6023	0.7412	26.183	27.425	64.8	0.85	P
C03	0.7319	0.5940	0.7353	36.296	37.834	122.6	0.49	P
C04	0.7310	0.5996	0.7282	41.973	43.176	123.0	0.26	P
R01	**0.7288**	**0.5941**	**0.7034**	**24.010**	**25.821**	**68.1**	**3.82**	**P**
R02	0.7330	0.5997	0.7108	42.963	44.721	129.1	1.26	P
R03	0.7265	0.5865	0.7281	32.386	34.875	128.2	1.85	F
Q01	0.5932	0.4743	0.6591	25.114	27.849	47.8	1.09	F
Q02	0.5965	0.4832	0.6504	25.183	27.597	43.4	1.32	F
Q04	0.0036	0.3702	0.4531	37.994	40.587	48.1	0.58	F
Q05	0.7358	0.5981	0.7102	43.333	45.668	111.8	1.06	P
Q06	0.0041	0.3541	0.4566	39.409	41.529	64.8	0.23	F
Q07	0.0282	0.3579	0.4289	38.299	40.826	53.0	0.89	F

P: accuracy gate passed; F: failed; D: S04 diagnostic, excluded from formal comparison. Bold candidate rows are nondominated. U01–U03 and S03 were not measured after static exclusion; Q03 failed engine construction. Latencies pool 600 timed iterations. CV uses the three repeat medians. Values are rounded for display; gate and Pareto decisions use unrounded values.

Table A2. Class-level accuracy of the baseline and final candidates

Class	B01			C01			R01		
	BBox AP	Mask AP	IoU	BBox AP	Mask AP	IoU	BBox AP	Mask AP	IoU
out_line	0.5061	0.2040	0.4589	0.5071	0.2022	0.4927	0.4797	0.1815	0.4461
parking_lot	0.8489	0.8086	0.7920	0.8622	0.8192	0.8369	0.8513	0.8052	0.7815
parking_space	0.8585	0.7954	0.8850	0.8465	0.7873	0.8805	0.8555	0.7957	0.8828

IoU denotes class-wise semantic intersection over union.

Data records [15]: results/stage2-notebook-summary.json consolidates these metrics. Accuracy and class-level results are in results/coco-evaluation/{ID}-front.json; raw latency repeats are under results/benchmarks/notebook-controlled/20260904_104755/{ID}/repeat-*/. Engine and annotation SHA-256 hashes identify the evaluated artifacts.